$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \mathbf{let}\ x = e_1\ \mathbf{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1 : \mathbf{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \mathbf{if}\ e_1\ \mathbf{then}\ e_2\ \mathbf{else}\ e_3 : \tau}$$

$$\textsc{Observe}\ \frac{\Gamma \vdash e : \mathbf{bool}}{\Gamma \vdash \mathbf{observe}\ e : \mathbf{unit}} \qquad \textsc{Discrete}\ \frac{}{\Gamma \vdash \mathbf{discrete}(p_0, \ldots, p_{n-1}) : \mathbf{fin}(n)} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \mathbf{fin}(n)}$$

$$\textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \mathbf{fin}(n) \qquad \Gamma \vdash e_2 : \mathbf{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \mathbf{bool}} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \mathbf{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the $\text{answers}_<(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_<(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {<}x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {\le}x \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates*. That is, if B contains ${<}x$ then the discretized value of $e : \mathbf{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \mathbf{float}[B; V_1]$ and $e_2 : \mathbf{float}[B; V_2]$, then $\text{answers}_<(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all ${<}x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

~~Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed~~When both comparands have unrestricted value sets, answerability requires $B = \top$: the comparison must remain over reals because neither operand can supply a finite set of thresholds. This does not assert $B = \top$ iff $V = \top$ for arbitrary float types; for example, a sampler can have $V = \top$ and a finite cut set. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\text{answers}_\le(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {\le}x \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {<}x \in B$.

Subtyping. The type system includes a subtyping relation that captures when one type can be replaced by another. The subtyping rules are shown in Figure 8. For float types $\mathbf{float}[B; V]$,

$$\textsc{Sub-Refl}\ \frac{}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\mathbf{float}[B_1; V_1] <: \mathbf{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'}$$

$$\textsc{Sub-List}\ \frac{\tau <: \tau'}{\mathbf{list}(\tau) <: \mathbf{list}(\tau')} \qquad \textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

subtyping follows the lattice ordering on V: an expression known to range over some set of values can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with ~~its most precise~~ a sound type. Specifically, we focus on expressions of type $\mathbf{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. The solver may conservatively add redundant cuts, so we do not claim that its annotations are always principal. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

The distinction between typing and type inference is important. The correctness argument cleanly separates into two pieces: (1) type inference produces a typing derivation matching our typing rules, and (2) given such a typing derivation, discretization is semantics preserving. Thus the proof of discretization does not have to reason directly about the type inference procedure.

4.1 Constraint Generation

We use the constraint language in Figure 9 to express requirements over the type structure and float refinements. We adopt the framework of [38], introducing a new judgment of the form $\Gamma \vdash e : \tau \mid C$, which reads as *in the environment* Γ, *e has type* τ *and produces the set of constraints* C. In this judgment, Γ and e are inputs to the type inference algorithm, while τ and C are outputs: the algorithm infers the type τ for expression e and generates the constraints C that must hold for this typing to be valid. The inference rules formalize which constraints must be generated and later solved for a given expression e to have inferred type τ. Figure 10 gives the constraint generation rules for Slice. We focus on a small but representative subset of constructs in Slice, capturing the key ideas behind constraint generation for the *full* language.

The type inference rules closely mirror the typing rules in Figure 6, with two key differences. First, the inference rules generate a constraint set instead of having those constraints appear in the premises. Second, subtyping is handled in a syntax-directed way: rather than using a general subtyping rule, we bake the subtyping relation into each rule.

Example 4.1. To make the constraint generation process concrete, Figure 11 walks through §2.3. Each subexpression is annotated with type variables for the cut sets (B_i) and value sets (V_i), and the constraints produced at each step are shown explicitly.

$$C ::= \tau_1 <: \tau_2 \mid \text{recovers}(B, V) \mid \text{answers}_<(B, V_1, V_2) \mid \text{answers}_\leq(B, V_1, V_2) \mid \text{has}(V, c)$$
$$\mid B_1 \sqsubseteq B_2 \mid B_1 = B_2 \mid V_1 \sqsubseteq V_2 \mid V_1 = V_2 \mid B_1 \equiv_\top B_2 \mid C_1, C_2$$

Fig. 9. Syntax for our constraint language. Constraints correspond to the semantic requirements in our type system: subtyping ensures type compatibility, $\text{recovers}(B, V)$ ensures that distribution parameters can be recovered after discretization, $\text{answers}_<(B, V_1, V_2)$ and $\text{answers}_\leq(B, V_1, V_2)$ ensure that comparison results can be determined after discretization, and $\text{has}(V, c)$ ensures that constants are properly tracked. The top-alignment constraint $B_1 \equiv_\top B_2$ holds exactly when $B_1 = \top \iff B_2 = \top$.

TFloat-Const
$$\frac{}{\Gamma \vdash c : \texttt{float}[B; V] \mid \text{has}(V, c)}$$

TIf
$$\frac{\Gamma \vdash e_1 : \texttt{bool} \mid C_1 \qquad \Gamma \vdash e_2 : \tau_2 \mid C_2 \qquad \Gamma \vdash e_3 : \tau_3 \mid C_3}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau_4 \mid C_1, C_2, C_3, \tau_2 <: \tau_4, \tau_3 <: \tau_4}$$

TVar
$$\frac{}{\Gamma, x : \tau \vdash x : \tau \mid \emptyset}$$

TLess
$$\frac{\Gamma \vdash e_1 : \texttt{float}[B_1; V_1] \mid C_1 \qquad \Gamma \vdash e_2 : \texttt{float}[B_2; V_2] \mid C_2}{\Gamma \vdash e_1 < e_2 : \texttt{bool} \mid C_1, C_2, \text{answers}_<(B, V_1, V_2) B_1 = B_2, \text{answers}_<(B_1, V_1, V_2)}$$

TContinuous-1
$$\frac{\Gamma \vdash e : \texttt{float}[B_1; V_1] \mid C}{\Gamma \vdash \texttt{exponential}(e) : \texttt{float}[B; V] \mid C, \text{recovers}(B_1, V_1), B_1 \equiv_\top B, V = \top}$$

TContinuous-2
$$\frac{\Gamma \vdash e_1 : \texttt{float}[B_1; V_1] \mid C_1 \qquad \Gamma \vdash e_2 : \texttt{float}[B_2; V_2] \mid C_2}{\Gamma \vdash \texttt{uniform}(e_1, e_2) : \texttt{float}[B; V] \mid C_1, C_2, \text{recovers}(B_1, V_1), B_1 \equiv_\top B, \text{recovers}(B_2, V_2), B_2 \equiv_\top B, V = \top}$$

TLessEq
$$\frac{\Gamma \vdash e_1 : \texttt{float}[B_1; V_1] \mid C_1 \qquad \Gamma \vdash e_2 : \texttt{float}[B_2; V_2] \mid C_2}{\Gamma \vdash e_1 \leq e_2 : \texttt{bool} \mid C_1, C_2, \text{answers}_\leq(B, V_1, V_2) B_1 = B_2, \text{answers}_\leq(B_1, V_1, V_2)}$$

TLet
$$\frac{\Gamma \vdash e_1 : \tau_1 \mid C_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2 \mid C_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2 \mid C_1, C_2}$$

Fig. 10. Constraint generation rules for type inference for float-typed expressions. All variables appearing only in a conclusion are fresh.

4.2 Constraint Solving

Once constraint generation completes, we obtain a system of constraints over type variables, cut sets B, and value sets V. The constraint-solving phase proceeds in two steps:

(1) **Subtyping reduction**: We first process all subtyping constraints of the form $\tau_1 <: \tau_2$. Since subtyping for float types follows the rule $\texttt{float}[B_1; V_1] <: \texttt{float}[B_2; V_2]$ if and only if $B_1 = B_2$ and $V_1 \sqsubseteq V_2$, we reduce each subtyping constraint to corresponding equality and subset constraints on the cut and value sets. Subtyping elimination continues until only constraints among B's and V's remain.

(2) **Lattice constraint solving**: After subtyping reduction, we are left with constraints purely over cut sets and value sets: equalities, subset relations, top-alignment, and the specialized constraints $\text{recovers}(B, V)$, $\text{answers}_<(B, V_1, V_2)$, $\text{answers}_\leq(B, V_1, V_2)$, and $\text{has}(V, c)$. These constraints are solved using a monotone fixed-point algorithm over the cut and value lattices.

Subtyping Reduction. Subtyping reduction eliminates all remaining type-level subtyping constraints by recursively decomposing them according to type structure. For base float types, a

Program	Constraints Emitted
`let x = if flip()`	
`then uniform(0 :float`$[B_1; V_1]$`,`	+ has$(V_1, 0)$
`2 :float`$[B_2; V_2]$`)`	+ has$(V_2, 2)$
`:float`$[B_3; V_3]$	+ recovers$(B_1, V_1), B_1 \equiv_\top B_3$, recovers$(B_2, V_2), B_2 \equiv_\top B_3, V_3 = \top$
`else gaussian(0 :float`$[B_4; V_4]$`,`	+ has$(V_4, 0)$
`1 :float`$[B_5; V_5]$`)`	+ has$(V_5, 1)$
`:float`$[B_6; V_6]$	+ recovers$(B_4, V_4), B_4 \equiv_\top B_6$, recovers$(B_5, V_5), B_5 \equiv_\top B_6, V_6 = \top$
`:float`$[B_7; V_7]$	+ **float**$[B_3; V_3]$ <: **float**$[B_7; V_7]$, **float**$[B_6; V_6]$ <: **float**$[B_7; V_7]$
`in`	
`let y = if x :float`$[B_7; V_7]$	+ $\emptyset$
`<`	
`1.5 :float`$[B_8; V_8]$	+ has$(V_8, 1.5)$
`:bool`	+ answers$_<(B_7, V_7, V_8), B_7 = B_8$
`then 1.8 :float`$[B_9; V_9]$	+ has$(V_9, 1.8)$
`else 0.3 :float`$[B_{10}; V_{10}]$	+ has$(V_{10}, 0.3)$
`:float`$[B_{11}; V_{11}]$	+ **float**$[B_9; V_9]$ <: **float**$[B_{11}; V_{11}]$, **float**$[B_{10}; V_{10}]$ <: **float**$[B_{11}; V_{11}]$
`in`	
`x :float`$[B_7; V_7]$	+ $\emptyset$
`<=`	
`y :float`$[B_{11}; V_{11}]$	+ $\emptyset$
`:bool`	+ answers$_\leq(B_7, V_7, V_{11}), B_7 = B_{11}$

Fig. 11. Program from §2.3 (left) and its emitted constraints from each line (right) from TFloat-Const, TContinuous-2, TIf, TVar, and TLess and TLessEq following the constraint generation rules in Figure 10. Lines annotated with $\emptyset$ emit no constraints.

constraint **float**$[B_1; V_1]$ <: **float**$[B_2; V_2]$ is reduced to the conjunction $B_1 = B_2$ and $V_1 \sqsubseteq V_2$. For compound types, subtyping is handled structurally before reaching floats. For example, a constraint **float**$[B_1; V_1] * $ **float**$[B_2; V_2]$ <: **float**$[B_5; V_5] * $ **float**$[B_6; V_6]$ is decomposed into **float**$[B_1; V_1]$ <: **float**$[B_5; V_5]$ and **float**$[B_2; V_2]$ <: **float**$[B_6; V_6]$, each of which is then reduced to equality and subset constraints on the corresponding cut and value sets. This process is syntactic and continues until no subtyping constraints remain, leaving only constraints over B and V.

Lattice Solving. The lattice solver keeps track of the current contents of all B and V variables. Initially, all B and V variables are set to the lattice bottom $\emptyset$; explicit $\top$ constraints are solver updates, not initial values. During the solving process, the variables may go up in the lattice to a finite set, or to $\top$. Each of the constraints is handled as follows:

- **Value containment** (has(V, c)): We update V to include the constant c, i.e., $V := V \sqcup \{c\}$. If V is already $\top$, the constraint is trivially satisfied. (1)
- **Top assignment** ($V = \top$): We immediately set V to $\top$, representing that the value set contains all possible float values. (2)
- **Subset constraints** ($B_1 \sqsubseteq B_2$ or $V_1 \sqsubseteq V_2$): We ensure that the contents of the first variable are always a subset of the second. When B_1 or V_1 is updated, we propagate the changes to B_2 or V_2 by taking the least upper bound. (3)
- **Recoverability** (recovers(B, V)): This constraint ensures that the cut set B can distinguish all values in V. If $B = \top$, the constraint is already satisfied. If $V = \top$ and $B \neq \top$, we set $B := \top$, since no finite cut set recovers an unrestricted real value. Otherwise V is finite; our implementation adds $\leq x$ for every $x \in V$. These cuts place distinct elements of V ~~is a finite set $\{v_1, ..., v_n\}$ with $n > 1$, we require that B contains at least $n - 1$ cuts to separate these values. If the cuts do not separate the values in the current solver state, we insert additional cuts to satisfy this condition. There is no canonical way to do this, so we use a heuristic: we insert a cut $< x$ for each value $x \in V$ that is not~~

~~already separated by existing cuts~~ in distinct intervals and therefore satisfy recoverability. This choice can add redundant cuts, but remains sound. ~~If $B = \top$, the constraint is trivially satisfied.~~ (4)

- **Less-than answerability** ($\text{answers}_<(B, V_1, V_2)$): Intuitively, this constraint ensures that B contains enough cuts to answer all possible comparisons of the form "$v_1 < v_2$" where $v_1 \in V_1$ and $v_2 \in V_2$. If $B = \top$, the constraint is trivially satisfied. Otherwise, ~~we again have a heuristic choice : we add a cut~~ the answerability definition gives a choice when both value sets are finite. The implementation satisfies the right-operand clause by adding $<v$ for each ~~value $v \in V_2$ (if $V_2 \neq \top$) and~~ $v \in V_2$; it does not also add the redundant left-operand cuts. If only V_1 is finite, it instead adds $\leq v$ for each ~~value $v \in V_1$ (if $V_1 \neq \top$). If V_1 and V_2 are both~~ $v \in V_1$. If both value sets are $\top$, ~~we set B to $\top$~~it sets $B := \top$. (5)
- **Less-equal answerability** ($\text{answers}_\leq(B, V_1, V_2)$): ~~Analogous to less-than~~The same case split uses $\leq v$ from the right operand when it is finite, $<v$ from the left operand only when the right operand is $\top$, and $B := \top$ when both are $\top$. (5)
- **Equality constraints** ($B_1 = B_2$ or $V_1 = V_2$): We unify the variables, treating them as the same variable throughout the solving process. Any update to one variable is immediately reflected in all variables unified with it. (6)
- **Top alignment** ($B_1 \equiv_\top B_2$): If either variable becomes $\top$, we set the other to $\top$. If both remain finite, the constraint is satisfied; unlike equality, this constraint does not propagate finite cuts. (7)

The solver repeatedly processes constraints until reaching a fixed point, where no constraint causes any further updates to the variables. ~~The monotonicity of the lattice operations ensures that this~~ Every update is monotone. A finite update draws cuts or values only from the finite sets already present in the initial environment or from constants in the generated constraints, while a $\top$ update is permanent. Thus a finite input judgment induces a finite-height reachable sublattice and the process terminates. This procedure may add more cuts to Bs than strictly necessary, but it never sets $B = \top$ unless that is ~~the only way to satisfy all constraints . In other words, the discretization itself may be suboptimal in terms of the number of cuts.~~ required by a $\top$ assignment, recoverability, answerability, top alignment, or propagation from such a constraint.

Theorem 4.2 (Displayed refinement-fragment soundness). *For a judgment generated by the rules in Figure 10 whose subtype constraints range only over base and float types, if subtyping reduction succeeds and lattice solving reaches a fixed point ρ, then ρ satisfies every generated constraint. Consequently, substituting ρ in that judgment yields a derivation using the corresponding displayed declarative rules.*

Proof. Float subtyping emits exactly cut equality and value inclusion. For the remaining constraints, inspect the update associated with each form above. Containment and $\top$ assignment hold after their direct updates. At a fixed point, a subset constraint is closed under join and an equality constraint under joins in both directions. Recoverability holds because either $B = \top$, or finite V has a $\leq$ cut at each of its values; the case $V = \top$ would otherwise still enable the $B := \top$ update. The answerability case split satisfies one of Definitions 3.3 and 3.4's finite clauses, or sets B to $\top$ when both value sets are $\top$. Finally, top alignment propagates $\top$ in both directions. Thus no constraint generated by Figure 10 is false at a fixed point. The displayed rules then give the corresponding declarative derivation by structural induction over this fragment. □

This theorem does not claim soundness for an otherwise unspecified full Hindley–Milner implementation. In particular, arrow-subtyping reduction, which must be contravariant in the domain and covariant

in the codomain, is outside the displayed fragment. A full pipeline theorem must also include the separate *dispatchable* obligations and the contextual-interface $\top$ updates: those updates must run before the final fixed point is checked, and their listeners must be allowed to resaturate the constraints. Those obligations are repaired independently in PR #4.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\mathbf{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\mathbf{float}[B; V] \to \mathbf{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.3. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression $e : \mathbf{bool}$ and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \mathbf{bool}]\!] : \mathbf{bool}$ such that e and $\mathcal{D}[\![e : \mathbf{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by Theorem 6.1: the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error bound. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization makes that reduction explicit.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 proves that this non-local, type-mediated construction preserves semantics. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \mathbf{uniform}(e_1, e_2)$ of type $\mathbf{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression $e' : \mathbf{fin}(n+1)$ such that the probability that e evaluates to a value in $\mathrm{Intervals}(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\mathbf{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized*.

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

where $B = \{<0.3, <0.5\}$. The cut set B informs us that the probability of the comparisons evaluating to true is determined by the probability of x evaluating to one of the intervals

$$\mathrm{Intervals}(B)_0 = (-\infty, 0.3) \qquad \mathrm{Intervals}(B)_1 = [0.3, 0.5) \qquad \mathrm{Intervals}(B)_2 = [0.5, \infty)$$

Subtyping Constraint	After Subtyping Reduction
$\texttt{float}[B_3; V_3] <: \texttt{float}[B_7; V_7]$	$B_3 = B_7,\ V_3 \sqsubseteq V_7$
$\texttt{float}[B_6; V_6] <: \texttt{float}[B_7; V_7]$	$B_6 = B_7,\ V_6 \sqsubseteq V_7$
$\texttt{float}[B_9; V_9] <: \texttt{float}[B_{11}; V_{11}]$	$B_9 = B_{11},\ V_9 \sqsubseteq V_{11}$
$\texttt{float}[B_{10}; V_{10}] <: \texttt{float}[B_{11}; V_{11}]$	$B_{10} = B_{11},\ V_{10} \sqsubseteq V_{11}$

(a) *Subtyping reduction.* Each subtyping constraint over float types is decomposed into an equality constraint on cut sets and a subset constraint on value sets.

	Constraints Processed	Solver Updates
		$\forall i\colon B_i = \emptyset, V_i = \emptyset$
①	$\text{has}(V_1, 0)$	$V_1 := V_1 \sqcup \{0\} = \{0\}$
	$\text{has}(V_2, 2)$	$V_2 := V_2 \sqcup \{2\} = \{2\}$
	$\text{has}(V_4, 0)$	$V_4 := V_4 \sqcup \{0\} = \{0\}$
	$\text{has}(V_5, 1)$	$V_5 := V_5 \sqcup \{1\} = \{1\}$
	$\text{has}(V_8, 1.5)$	$V_8 := V_8 \sqcup \{1.5\} = \{1.5\}$
	$\text{has}(V_9, 1.8)$	$V_9 := V_9 \sqcup \{1.8\} = \{1.8\}$
	$\text{has}(V_{10}, 0.3)$	$V_{10} := V_{10} \sqcup \{0.3\} = \{0.3\}$
②	$V_3 = \top$	$V_3 := \top$
	$V_6 = \top$	$V_6 := \top$
③	$V_3 \sqsubseteq V_7$	$V_7 := V_7 \sqcup \top = \top$
	$V_6 \sqsubseteq V_7$	$\emptyset$
	$V_9 \sqsubseteq V_{11}$	$V_{11} := V_{11} \sqcup \{1.8\} = \{1.8\}$
	$V_{10} \sqsubseteq V_{11}$	$V_{11} := V_{11} \sqcup \{0.3\} = \{0.3, 1.8\}$
④	$\text{recovers}(B_1, V_1)$	$\emptyset$
	$\text{recovers}(B_2, V_2)$	$\emptyset$
	$\text{recovers}(B_4, V_4)$	$\emptyset$
	$\text{recovers}(B_5, V_5)$	$\emptyset$
⑤	$\text{answers}_<(B_7, V_7, V_8)$	$B_7 := B_7 \sqcup \{<1.5\} = \{<1.5\}$
	$\text{answers}_\leq(B_7, V_7, V_{11})$	$B_7 := B_7 \sqcup \{\leq 0.3, \leq 1.8\}$ $= \{<1.5, \leq 0.3, \leq 1.8\}$
⑥	$B_7 = B_8, B_7 = B_{11}$	$B_7 = B_8, B_7 = B_{11}$
	$B_3 = B_7, B_6 = B_7$	$B_3 = B_7, B_6 = B_7$
	$B_9 = B_{10}, B_{10} = B_{11}$	$B_9 = B_{11}, B_{10} = B_{11}$
⑦	$B_1 \equiv_\top B_3, B_2 \equiv_\top B_3$	$\emptyset$
	$B_4 \equiv_\top B_6, B_5 \equiv_\top B_6$	$\emptyset$

(b) *Lattice solving.* Each pass performs monotone updates to B_i or V_i. Lines annotated with $\emptyset$ denote constraint is trivially satisfied.

Cut Sets

$B_1 = \emptyset$
$B_2 = \emptyset$
$B_4 = \emptyset$
$B_5 = \emptyset$
$B_3 = B_6$
$= B_7 = B_8$
$= B_9 = B_{10}$
$= B_{11}$
$= \{<1.5, \leq 0.3, \leq 1.8\}$

Value Sets

$V_1 = \{0\}$
$V_2 = \{2\}$
$V_3 = \top$
$V_4 = \{0\}$
$V_5 = \{1\}$
$V_6 = \top$
$V_7 = \top$
$V_8 = \{1.5\}$
$V_9 = \{1.8\}$
$V_{10} = \{0.3\}$
$V_{11} = \{0.3, 1.8\}$

(c) Final cut sets and value sets obtained.

Fig. 12. Following Figure 11, the constraint solving process for §2.3.